

Использование языка программирования Julia для решения математических задач

Часть II

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Алексеев, Е. Р. Scilab: Решение инженерных и математических задач: учеб. издание. / Е.Р. Алексеев, К. В. Дога, О. В. Чеснокова М. – Базальт СПО; ДМК Пресс, 2024. – 440 с.
2. Балакин, А. А. Численные методы и математическое моделирование: Учебное пособие / А.А. Балакин. – Долгопрудный: Издательский Дом «Интеллект», 2022. – 288 с.
3. Деза, Е. И. Введение в криптографию: Теоретико-числовые основы защиты информации. Учебное пособие / Е. И. Деза, Л. В. Котова. – Москва: ЛЕНАНД, 2022. – 376 с.
4. Кнауб, Л. В. Теоретико-численные методы в криптографии / Л. В. Кнауб, Е. А. Новиков, Ю. А. Шитов; Сибирский федеральный университет – Красноярск: 2011. – 160 с.

Глава 16 Криптография

Криптография – это наука и искусство обеспечения безопасности информации. Она охватывает методы шифрования, которые позволяют преобразовать читаемую информацию в недоступный для понимания вид, обеспечивая таким образом защиту данных от несанкционированного доступа.

Шифрование – это процесс преобразования открытого текста (исходной информации) в зашифрованный текст с помощью криптографического алгоритма.

Дешифрование – это процесс восстановления исходной (открытой) информации из зашифрованного текста.

16.1 Пример шифрования с использованием Unicode

Unicode – это стандарт кодирования, который позволяет представлять символы различных языков, включая латиницу, кириллицу, иероглифы и множество других символов. Каждому символу в Unicode соответствует уникальный числовой код.

Для начала рассмотрим самый простой способ шифрования, который заключается в том, чтобы преобразовать символы в их код Unicode. В этом методе каждый символ текста будет просто преобразован в его числовое представление, а затем при необходимости обратно в символ. Это будет

простая форма шифрования, которая поможет нам понять, как работает кодировка.

```
In [41]: #Пример шифрования
function string_to_int(message::String)
    result = BigInt(0)          # Инициализируем результат как большое целое
    base = BigInt(65536)        # Основание для кодирования (65536 - 16 бит)
    #Для большинства символов в Unicode можно использовать 16-битное представление

    for c in message
        char_code = Int(c)      # Получаем Unicode код символа
        result = result * base + BigInt(char_code) # Добавляем код символа
    end

    return result
end

message = "Привет"
encoded = string_to_int(message)
println("Шифрование сообщения: ", encoded)
```

Шифрование сообщения: 1275436810054983563020141634

```
In [42]: #Пример дешифрования
function int_to_string(num::BigInt)
    result = ""
    base = BigInt(65536) # Основание для Unicode кодов (16 бит)

    while num > 0
        char_code = Int(num % base) # Извлекаем код символа
        result = Char(char_code) * result # Преобразуем код в символ и добавляем
        num ÷= base # Разделяем число на основание
    end

    return result
end

# Пример использования
encoded_number = BigInt(1275436810054983563020141634)
decoded_string = int_to_string(encoded_number)
println("Дешифрование сообщения: ", decoded_string)
```

Дешифрование сообщения: Привет

Ключ шифрования – это параметр или секретная последовательность символов, используемая для выполнения криптографической операции (шифрования или дешифрования) с целью защиты данных.

Каждый криптографический алгоритм использует ключи по-разному. Ключ шифрования может быть числом, строкой или даже более сложной структурой данных, в зависимости от типа алгоритма.

Современная криптография делится на два типа: **симметричную и асимметричную**.

Симметричное шифрование – это метод шифрования данных, при котором используется один и тот же ключ для шифрования и дешифрования информации.

Асимметричное шифрование – это метод шифрования, при котором используются два различных ключа: открытый ключ для шифрования и закрытый ключ для дешифрования.

Открытый ключ – это часть ключевой пары, доступная для всех пользователей системы. Он используется для шифрования данных и может быть публично распространён, в отличие от закрытого ключа.

Закрытый ключ – это секретная часть ключевой пары, которая используется для дешифрования данных, зашифрованных с использованием открытого ключа. Закрытый ключ должен храниться в безопасности и не должен передаваться третьим лицам.

Давайте рассмотрим пример, где в качестве ключа для шифрования мы будем использовать сдвиг Unicode. Это значит, что для каждого символа в строке мы будем сдвигать его код (по стандарту Unicode) на некоторое фиксированное количество позиций, которое и будет нашим ключом.

```
In [43]: # Функция для шифрования сообщения с использованием сдвига Unicode
function shift_encrypt(message::String, key::Int)
    result = BigInt(0)
    base = BigInt(65536)
    for c in message
        # Получаем код символа Unicode
        char_code = Int(c) + key # Сдвигаем код символа на величину ключа
        result = result * base + BigInt(char_code)
    end
    return result
end

message = "Привет" # Исходное сообщение
key = 5 # Ключ сдвига

# Шифруем сообщение
encrypted_message = shift_encrypt(message, key)
println("Зашифрованное сообщение: ", encrypted_message)
```

Зашифрованное сообщение: 1281481531388184473800148039

Чтобы расшифровать зашифрованное сообщение, нужно знать ключ – величину сдвига, которую мы использовали при шифровании.

```
In [44]: #Пример дешифрования
function int_to_string(num::BigInt, key::Int)
    result = ""
```

```

base = BigInt(65536) # Основание для Unicode кодов (16 бит)

while num > 0
    char_code = Int(num % base - key) # Извлекаем код символа
    result = Char(char_code) * result # Преобразуем код в символ и доба
    num ÷= base # Разделяем число на основание
end

return result
end

# Пример использования
encoded_number = BigInt(1281481531388184473800148039)
key = 5

decoded_string = int_to_string(encoded_number, key)
println("Дешифрование сообщения: ", decoded_string)

```

Дешифрование сообщения: Привет

16.2 Шифр Цезаря

Теперь рассмотрим шифр Цезаря, который похож на то, что мы рассматривали ранее с сдвигом Unicode, но в отличие от этого шифр Цезаря применяет сдвиг по буквам в алфавите.

Шифр Цезаря – это классический метод шифрования, в котором каждый символ текста сдвигается на некоторое фиксированное количество позиций в алфавите. Например, если мы используем сдвиг на 3, то буква "А" станет "Г", "Б" станет "Д", и так далее.

```

In [9]: # Функция для шифрования с использованием шифра Цезаря
function caesar_encrypt(message::String, key::Int)
    encrypted_message = "" # Инициализируем пустую строку для хранения зашифрованного сообщения

    # Мы перебираем каждый символ char в исходном сообщении message
    for char in message
        if islowercase(char) # Проверяется, является ли символ строчной буквой
            base = 'a' # Если символ – строчная буква, то используем 'a' как базовый символ
            encrypted_char = Char(mod((Int(char) - Int(base) + key), 32) + Int(base))
            # Int(char) - Int(base): Переводим символ в число, вычитая код символа 'a'
            # (Int(char) - Int(base) + key): Добавляем сдвиг shift к числовому значению
            # mod(..., 32): Оператор mod используется для того, чтобы после сдвига
            # (например, после буквы 'я') снова вернуться к началу алфавита.
            # + Int(base): После применения сдвига возвращаемся к коду символа 'a'
            encrypted_message *= encrypted_char # Добавляем зашифрованный символ к строке
        elseif isuppercase(char) # Если символ является прописной буквой
            base = 'A' # Устанавливаем базовый символ для прописного алфавита
            encrypted_char = Char(mod((Int(char) - Int(base) + key), 32) + Int(base))
            # Преобразуем символ в число, сдвигаем его на 'key' позиций,
            # и потом переводим обратно в символ, учитывая ограничение длины алфавита
            encrypted_message *= encrypted_char # Добавляем зашифрованный символ к строке
        else
            encrypted_message *= char # Если символ не буква, то оставляем его без изменений
        end
    end
end

```

```

        # Если символ не является буквой (например, пробел, знак препинания)
        encrypted_message *= char # Оставляем его без изменений
    end
end

return encrypted_message # Возвращаем зашифрованное сообщение
end

# Функция для дешифрования с использованием шифра Цезаря
function caesar_decrypt(encrypted_message::String, key::Int)
    return caesar_encrypt(encrypted_message, -key) # Для дешифрования сдвиг
end

# Пример использования
println("Введите сообщение")
message = readline()
println("Введенное сообщение: ", message)

println("Введите ключ")
key = parse{Int, readline()}
println("Введенный ключ: ", key)

# Шифруем сообщение
encrypted_message = caesar_encrypt(message, key)
println("Зашифрованное сообщение: ", encrypted_message)

# Дешифруем сообщение
decrypted_message = caesar_decrypt(encrypted_message, key)
println("Дешифрование сообщения: ", decrypted_message)

```

```

Введите сообщение
Введенное сообщение: Привет
Введите ключ
Введенный ключ: 4
Зашифрованное сообщение: Уфмжйц
Дешифрование сообщения: Привет

```

16.3 Шифр Атбаш

Шифр Атбаш – это один из самых простых шифров, в котором буквы алфавита заменяются на противоположные. Например, 'А' заменяется на 'Я', 'Б' на 'Ю' и так далее.

```

In [54]: function atbash_cipher(message::String)
    encrypted_message = ""
    for char in message
        if 'А' <= char <= 'Я'
            # Для заглавных букв
            encrypted_char = Char{Int('А') + Int('Я') - Int(char)}
            encrypted_message *= encrypted_char
        elseif 'а' <= char <= 'я'
            # Для строчных букв
            encrypted_char = Char{Int('а') + Int('я') - Int(char)}

```

```

        encrypted_message *= encrypted_char
    else
        # Оставляем символ без изменений
        encrypted_message *= char
    end
end
return encrypted_message
end

message = "Пример использования"
println("Исходное сообщение: ", message)

# Шифруем сообщение
encrypted_message = atbash_cipher(message)
println("Зашифрованное сообщение: ", encrypted_message)

# Дешифруем сообщение
decrypted_message = atbash_cipher(encrypted_message)
println("Дешифрованное сообщение: ", decrypted_message)

```

Исходное сообщение: Пример использования
 Зашифрованное сообщение: Рпчуъп чорсфгшсэятча
 Дешифрованное сообщение: Пример использования

16.4 Алгоритм XOR-шифрования

XOR-шифрование – это один из самых простых, но в то же время мощных алгоритмов симметричного шифрования. Он использует операцию XOR (исключающее ИЛИ), которая работает с отдельными битами данных.

Основная идея XOR-шифрования заключается в том, что для шифрования и дешифрования используется одна и та же операция – побитовая операция XOR. Благодаря этому алгоритм обладает свойством симметричности: для расшифровки зашифрованного сообщения достаточно снова применить XOR с тем же самым ключом.

Что такое операция XOR?

XOR (исключающее ИЛИ) – это логическая операция, которая действует поразрядно, сравнивая два числа побитово. Суть этой операции заключается в том, что она возвращает **1**, если соответствующие биты двух чисел различны, и **0**, если они одинаковы.

Таблица истинности для операции XOR:

x	y	x $\oplus$ y
0	1	1
0	0	0
1	1	0

x	y	x $\underline{\vee}$ y
1	0	1

Чтобы выполнить операцию **XOR** в Julia, используется оператор $\underline{\vee}$. Ниже приведены примеры использования этой операции.

```
In [57]: # Функция для выполнения XOR и вывода результата в двоичном формате
function xor_binary(x::UInt8, y::UInt8)
    # Выполнение операции XOR
    result = x  $\underline{\vee}$  y

    # Выводим результат в двоичной строке
    println("Число x в двоичной системе: ", bitstring(x)) #функцию bitstring
    println("Число y в двоичной системе: ", bitstring(y))
    println("Результат XOR в двоичной системе: ", bitstring(result))

    return result
end

x = 0b10101010 # 0b – это префикс, который говорит компилятору, что дальше
y = 0b11001100 #

# Выполнение XOR и вывод результатов
xor_binary(x, y)
```

Число x в двоичной системе: 10101010

Число y в двоичной системе: 11001100

Результат XOR в двоичной системе: 01100110

Out[57]: 0x66

Основные шаги XOR-шифрования:

1. сообщение (строка) представляется в виде последовательности байтов. Каждый символ строки преобразуется в его числовое представление;
2. ключ также преобразуется в числовое представление. Если длина ключа меньше длины сообщения, ключ расширяется (например, повторяется несколько раз), чтобы его длина соответствовала длине сообщения;
3. для каждого символа сообщения применяется операция XOR;
4. результатом XOR-операции является зашифрованный текст, который состоит из зашифрованных байтов. Эти байты могут быть преобразованы обратно в строку для отображения зашифрованного сообщения;
5. для расшифровки сообщения нужно снова применить ту же операцию XOR с тем же ключом. Поскольку операция XOR является обратимой,

зашифрованное сообщение будет расшифровано обратно в исходное.

```
In [16]: # Функция для XOR-шифрования и дешифрования
function xor_encrypt_decrypt(message::String, key::String)
    # Преобразуем ключ в байтовое представление (массив чисел)
    key_bytes = [Int(c) for c in key]
    message_bytes = [Int(c) for c in message]

    key_extended = repeat(key_bytes, outer = ceil(Int, length(message) / length(key_bytes)))
    #repeat(key_bytes, outer = ...) повторяет массив key_bytes, создавая новый массив
    #который содержит несколько копий оригинального массива ключа
    #Аргумент outer = ceil(Int, length(message) / length(key_bytes)) рассчитан
    #чтобы его длина стала хотя бы равной длине сообщения. ceil(Int, ...) округляет
    key_extended = key_extended[1:length(message)]
    #key_extended = key_extended[1:length(message)] обрезает расширенный массив
    #чтобы длина key_extended точно совпала с длиной сообщения.

    # Применяем операцию XOR к каждому символу
    encrypted_message = Char{Char}(message_bytes[i] ⊕ key_extended[i]) for i in 1:length(message)

    return String(encrypted_message) # Преобразуем массив Char обратно в строку
end

println("Введите сообщение")
message = readline()
println("Введенное сообщение: ", message)
println("Введите ключ:")
key = readline()
println("Введенный ключ: ", key)

# Шифруем сообщение
encrypted_message = xor_encrypt_decrypt(message, key)
println("Зашифрованное сообщение: ", encrypted_message)

# Дешифруем сообщение
decrypted_message = xor_encrypt_decrypt(encrypted_message, key)
println("Дешифрованное сообщение: ", decrypted_message)
```

```
Введите сообщение
Введенное сообщение: XOR-шифрование
Введите ключ:
Введенный ключ: 45
Зашифрованное сообщение: lzfrЩЙΨvНЇЄJKĖ
Дешифрованное сообщение: XOR-шифрование
```

16.5 Алгоритм RSA

Алгоритм **RSA** (от имени авторов **R**ivest, **S**hamir, и **A**dleman) является одним из самых популярных и широко используемых асимметричных алгоритмов шифрования. В отличие от симметричных алгоритмов, которые используют один и тот же ключ для шифрования и дешифрования, RSA использует пару ключей: **открытый** и **закрытый**.

Принцип работы RSA.

1. Генерация ключей:

- выбираются два больших простых числа p и q (для выбора подходящих простых чисел часто используется тест Миллера-Рабина подробнее описанный в п.12.4.4);
- вычисляется их произведение $r = p \times q$, которое используется для формирования открытого и закрытого ключей;
- вычисляется функция Эйлера $\varphi(r) = (p - 1) \cdot (q - 1)$;
- открытый ключ e выбирается таким образом, чтобы он был взаимно прост с $\varphi(r)$ и $1 < e < \varphi(r)$ (то есть, $\gcd(e, \varphi(r)) = 1$) (для нахождения e используем стандартный алгоритм Евклида п. 12.2.1);
- закрытый ключ d должно удовлетворять условию $d \times e \equiv 1 \pmod{\varphi(r)}$ (для нахождения d можно использовать расширенный алгоритм Евклида п. 12.2.2).

2. Шифрование:

- для шифрования используется открытый ключ (e, r) ;
- сообщение M преобразуется в число m , которое должно быть меньше r ;
- шифрование выполняется по формуле:

$$c = m^e \pmod{n}, \quad \text{где } c - \text{это зашифрованное сообщение}$$

3. Дешифрование:

- для дешифрования используется закрытый ключ (d, r) ;
- дешифрование выполняется по формуле:

$$m = c^d \pmod{n}$$

- полученное число m затем можно преобразовать обратно в сообщение.

In [3]:

```
using Random
using Dates

# Функция для быстрого возведения в степень по модулю
function modd(base, exp, mod)
    result = BigInt(1)
    base = base % mod
    while exp > 0
        if exp % 2 == 1
            result = (result * base) % mod
        end
        exp ÷= 2
    end
end
```

```

        base = (base * base) % mod
    end
    return result
end

# Тест Миллера-Рабина
function is_prime(n)
    iterations = 25
    if n < 2
        return false # Числа < 2 не простые
    end
    if n == 2
        return true # 2 - простое число
    end
    if n % 2 == 0
        return false # Чётные числа больше 2 не простые
    end

    d = n - 1
    r = BigInt(0)
    # Найти d такое, что n - 1 = 2^r * d
    while d % 2 == 0 # Пока d четное, делим d на 2 и увеличиваем r
        d ÷= 2
        r += 1
    end

    for i in 1:iterations
        a = rand(2:n-2) # Генерируем случайное a < n - 1
        x = modd(a, d, n) # x = a^d mod n
        if x != 1 && x != n - 1 # Проверка, не равен ли x 1 или n-1
            composite = true # Предполагаем, что n составное
            temp_r = r
            while temp_r > 0 # Цикл для проверки значений x
                x = modd(x, BigInt(2), n) # x = x^2 mod n
                if x == n - 1
                    composite = false # n простое
                    break # Выходим из цикла
                end
                temp_r -= 1
            end

            # Если x не стало равно n-1, число составное
            if composite
                return false
            end
        end
    end
    return true
end

function string_to_int(message::String)
    result = BigInt(0)
    base = BigInt(65536) # 65536 для Unicode кодов (с использованием 16 бит

    for c in message

```

```

        # Получаем Unicode код символа
        char_code = Int(c)
        result = result * base + BigInt(char_code) # Добавляем код символа
    end
    return result
end

function int_to_string(num::BigInt)
    result = ""
    base = BigInt(65536)

    while num > 0
        char_code = Int(num % base) # Получаем код символа
        result = Char(char_code) * result # Преобразуем в символ и добавляем
        num ÷= base # Делим число на основание 65536
    end

    return result
end

# Алгоритм Евклида
function euclid(a, b)
    while b != 0
        a, b = b, a % b
    end
    return a == 1
end

# Расширенный алгоритм Евклида
function advanced_euclid(a, b)
    x0, x1 = 1, 0
    r0, r1 = a, b
    while r1 != 0
        q = r0 ÷ r1
        r0, r1 = r1, r0 % r1
        x0, x1 = x1, x0 - q * x1
    end
    return x0 > 0 ? x0 : x0 + b
end

# Функция для генерации случайного простого числа длины n
function generate_random(length)
    while true
        startt = BigInt(10)^(length-1)
        endd = BigInt(10)^length - 1
        n = rand(startt:endd)
        if is_prime(n)
            return n
        end
    end
end

# Открытый ключ
function public_key(phi_r, length)
    while true

```

```

        startt = BigInt(10)^(length-1)
        endd = BigInt(10)^length - 1
        e = rand(startt:endd)
        if euclid(e, phi_r)
            return e
        end
    end
end

# Закрытый ключ
function private_key(e, phi_r)
    d = advanced_euclid(e, phi_r)
    return d
end

length_p = rand(90:100) #длина простого числа p
length_q = rand(90:100) #длина простого числа q
length_e = rand(90:100) #длина ключа e

println("Введите текст")
text = readline()

start_time = time()

# Генерация простых чисел p и q
p = generate_random(length_p)
q = generate_random(length_q)
r = p * q
phi_r = (p - 1) * (q - 1)

# Генерация открытого и закрытого ключей
e = public_key(phi_r, length_e)
d = private_key(e, phi_r)

# Преобразование текста в число
message_int = string_to_int(text)

# Шифрование
m_1 = modd(message_int, e, r)

# Дешифрование
m_2 = modd(m_1, d, r)

# Преобразование числа обратно в строку
decrypted_message = int_to_string(m_2)

# Печать результатов
end_time = time()
elapsed_time = end_time - start_time

println("Сгенерированное простое число p: $p")
println("Сгенерированное простое число q: $q")
println("Перемножение r: $r")
println("Функция Эйлера  $\phi(r)$ : $phi_r")
println("Открытый ключ e: $e")

```

```
#println("Закрытый ключ d: $d")
println("Текст: $text")
println("Шифрование: $m_1")
println("Дешифрование: $m_2")
println("Дешифрованный текст: $decrypted_message")
println("Время выполнения: $(round(elapsed_time, digits=2)) секунд")
```

Введите текст

Текст: Кубанский государственный Университет

Шифрование: 5299961254891204026178146863321024910670509214181865357371683517
2075358990127081061915125579763569331159948182473805296533920268481438993127
189838145065403349660838871307570822866148374574847

Дешифрование: 25970103900155606465954044589100686993596261069025937743698152
5379432630513968876127862588611028182740716515937292180316877133780508334056
777169885623964023366310519253199291458

Дешифрованный текст: Кубанский государственный Университет

Время выполнения: 0.19 секунд